

Introduction to RSS extension for Key Wrap (RSSe KW) service on STM32 MCUs

Introduction

This application note introduces the RSS extension Key Wrap (RSSe KW) feature available on the STM32 MCUs listed in Table 1.

In a context of symmetric or asymmetric cryptography, protection of secret keys is essential. For this purpose, ST proposes a solution so that the keys remain secret at any time and usable only on a specific device.

On STM32, two elements are required to do so. The first one is a hardware coupling and chaining bridge (CCB) to isolate the key manipulation inside hardware cryptographic IPs from any master on the system. The second one is a Root Security Services extension (RSSe) library allowing the user firmware to use the secret key while never giving access to the secret key itself. In the document, the library is referenced as “RSSe KW” and the output is referenced as “wrapped Key”.

The provisioning of unique keys may be required for some use cases as initial attestation, for example. This provisioning is a crucial step and may be very costly to guaranty the unicity and confidentiality of the keys. STMicroelectronics addresses this challenge by provisioning, in secure facilities, each applicable STM32 (cf. Table 1) unique key pairs called Device Unique Authentication keys (DUA). These keys are ECC (Elliptic Curve Cryptographic) keys of 256 bits. The public part is freely accessible through a certificate signed by an STMicroelectronics Certificate Authority (CA) while the wrapped private part is accessible only through the RSSe KW.

This application note gives an overview of the STM32 RSSe KW solution with its associated tools ecosystem and explains how to use it to safely manipulate secret keys for cryptographic operations.

Table 1. Applicable products

Type	Products
Microcontroller	STM32U366VE, STM32U366RE, STM32U366KE, STM32U366CE
	STM32U385CG, STM32U385RG, STM32U385VG, STM32U385KG, STM32U385RC, STM32U385RE
	STM32U3C5ZI, STM32U3C5ZH, STM32U3C5WI, STM32U3C5VI, STM32U3C5RI, STM32U3C5QI, STM32U3C5JI, STM32U3C5CI
	STM32H553ZG, STM32H553VG, STM32H553UG, STM32H553RG, STM32H553CG
	STM32H5F5ZJ, STM32H5F5VJ, STM32H5F5LJ, STM32H5F5IJ, STM32H5F5AJ, STM32H5F4ZJ, STM32H5F4VJ, STM32H5F4IJ, STM32H5F4AJ

Note: In this document, STM32U385xx, STM32U3C5xx, and STM32U366xx devices are referred as STM32U3.

Note: In this document, STM32H553xx and STM32H5Fxx devices are referred as STM32H5

1 General information

This document applies to the STM32U385xx, STM32U3C5xx, STM32U366xx, STM32H553xx, and STM32H5Fxx devices Arm® Cortex®-M33-based microcontrollers.

Note: *Arm and Cortex are registered trademarks of Arm Limited (or its subsidiaries or affiliates) in the US and/or elsewhere.*
Arm and TrustZone are registered trademarks of Arm Limited (or its subsidiaries or affiliates) in the US and/or elsewhere.
The Arm word and logo are trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere. All rights reserved.

1.1 References

Table 2. List of documents

Reference	Name	Title
[1]	AN3155	USART protocol used in the STM32 bootloader
[2]	AN3156	USB DFU protocol used in the STM32 bootloader
[3]	AN4221	I2C protocol used in the STM32 bootloader
[4]	AN4286	SPI protocol used in the STM32 bootloader
[5]	AN5927	I3C protocol used in the STM32 bootloader
[6]	RM0487	STM32U3 series Arm®-based 32-bit MCUs
[7]	RM0517	STM32H5E4/H5F4 and STM32H5E5/H5F5 reference manual
[8]	RM0539	STM32H523/33xx, STM32H562/63xx, and STM32H573xx reference manual
[9]	UM2237	STM32CubeProgrammer software description
[10]	UM2238	STM32 Trusted Package Creator tool software description

1.2 Glossary

Acronym	Definition
AES	Advanced Encryption Standard
AES CBC	AES Cipher Block Chaining mode
AES ECB	AES Electronic Code Block mode
AES GCM	AES Galois Counter Mode
CA	Certificate Authority
DUA	Device Unique Authentication
ECC	Elliptic Curve Cryptography
HDP	Hide Protection
KW	Key Wrapping
MAC	Message Authentication Code
MCU	Microcontroller Unit
OB	Option Byte
RDP	Readout Protection
RSS	Root Security Services

Acronym	Definition
RSSe KW	RSS extension Key Wrap
RSSe SFI	RSS extension Secure Firmware Install
Secure Boot	Root of Trust, checks STM32 security protection
Secure bootloader	Standard ST bootloader with additional security features
STM32 TPC	STM32 Trusted Package Creator (see document [10])
TZEN	TrustZone® enable option byte

2 STM32 RSSe Key Wrapping

2.1 RSSe KW principles overview

RSSe KW is a library implemented in STM32 microcontrollers that get wrapped keys in trusted facilities. The RSSe KW can be used through debugger access. The main goal of the RSSe KW is to provide wrapped keys. Here wrapped means usable only on one specific device.

The RSSe KW prevents the sensitive keys from:

- Being accessed by any firmware or debugging access
- Being used by another device (including STM32)

For the moment, the main goal of the RSSe KW is to provide wrapped DUA keys usable only on the current chip for a context given by the user. The DUA keys are provisioned in a trusted environment in ST facilities.

2.2 Wrapped key usage constraints

The output of the RSSe KW contains the key wrapped for a specific context along with elements needed to use it. On STM32U3 devices, the "specific context" depends on:

- the security state (secure or nonsecure)
- the privilege state (privileged only)
- the key usage (for ECC keys: signature or scalar multiplication computation)
- the product life cycle (RDP $\neq$ 0xAA)

On STM32H5 devices, the "specific context" depends on:

- The security state (secure)
- The privilege state (privileged only)
- The key usage (for ECC keys: signature or scalar multiplication computation)
- The HDP level (HDPL2 or HDPL3)
- The epoch counter selection: epoch force to zero, epoch secure if the security state is secure, or epoch nonsecure if the security state is nonsecure
- The product life cycle : Product State $\neq$ Open

All these elements can be chosen when launching the key wrap command except for the product life cycle.

Caution: *The output wrapped key can only be used on the specific device it has been requested. Moreover, it can only be used in the context requested during the service call. This allows the user to restrict the usage of the key to the context needed.*

2.3 RSSe KW features

2.3.1 Internal key wrapped

The STM32U3 and STM32H5 devices embeds two unique key pairs called the DUA. These pairs are provisioned at ST level in trusted facilities. The public part is accessible to the user through a certificate signed by a STMicroelectronics CA. The private part is never accessible by a user application nor a debugger access. The RSSe KW aims to provide wrapped DUA private keys. The private key is provided inside a container that contains all necessary informations to be used by the CCB in the customer's application. For more details about the container format, refer to [Section 3.3.5: RSSe_KEYWRAP_WrapEccCpvkKey](#).

2.4 RSSe KW constraints

The RSSe KW runs only in TZEN = 1 and PROVISIONING product state on STM32H5 devices.

The RSSe KW runs only in TZEN = 1 and RDP = 0x55 on STM32U3 devices.

The RSSe KW only runs on crypto parts.

One shot service: once installed, the RSSe KW allows only one service to be called (meaning one wrapped key structure at a time). To get another wrapped key on STM32U3 devices, the user must uninstall the RSSe KW (by doing a reset on the board), then, critically, do a RDP regression (RDP = 0xAA) and reinstall it, thus run the procedure from the beginning (as detailed later). To get another wrapped key, on STM32H5 devices, the user must uninstall the RSSe KW (by doing a reset on the board) before reinstalling it.

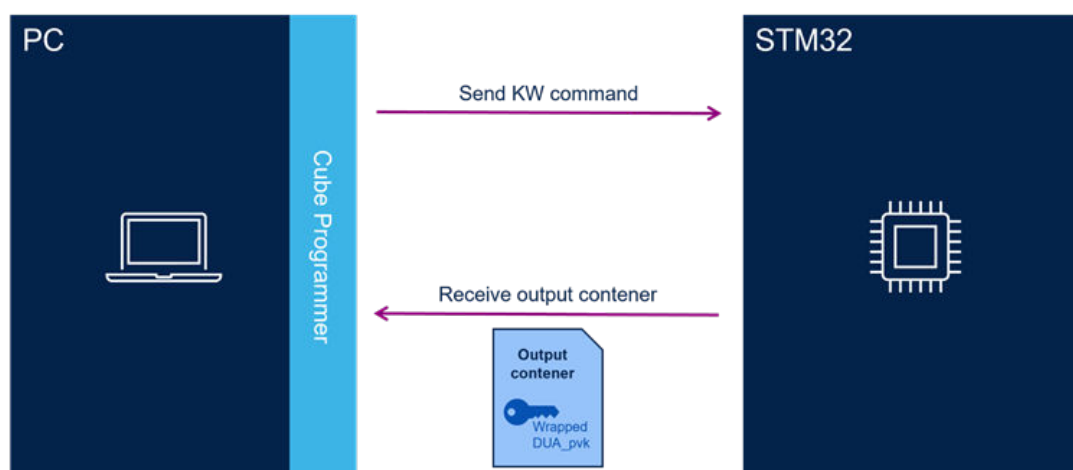
3 Interface

3.1 Cube Programmer interface

The Cube Programmer command to get wrapped DUA keys from the device is detailed in Cube Programmer documentation [9].

With the use of Cube Programmer, the user can get the wrapped key with only one command as shown in Figure 1. The output container is stored into a binary file chosen by the user.

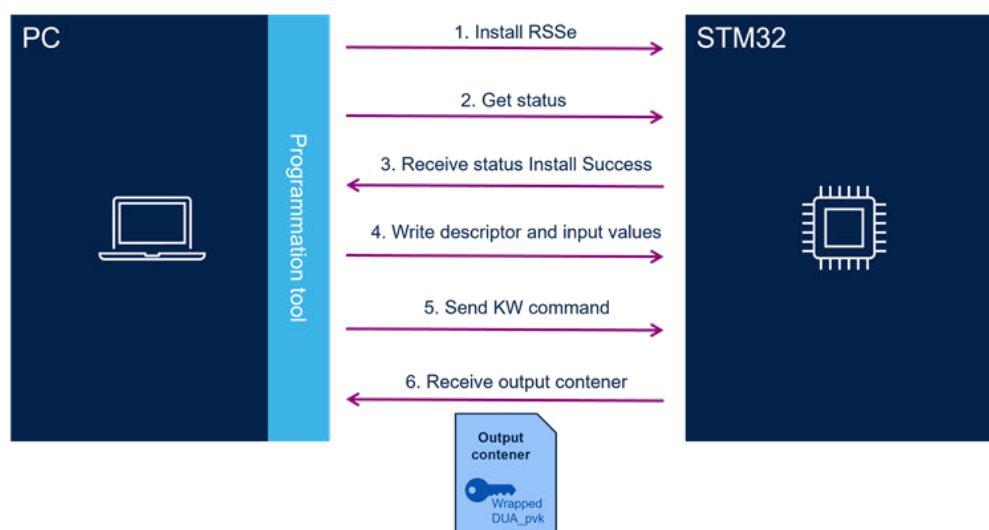
Figure 1. Cube Programmer interface with user



DT76293V1

With a custom tool, the user must follow the steps detailed in Figure 2 to get the wrapped key. This flow is also detailed in Section 3.3.5.

Figure 2. Detailed programming tool interface with user



DT76294V1

3.2 Tool/ RSSLIB interface

In case Cube Programmer cannot be used to get the wrap key, the programming tool must follow the steps below.

3.2.1 Key wrap flow overview

The user needs to be equipped with a programming tool. To get a specific wrapped key, the tool must perform the following steps:

1. Install the RSSe KW.
 - a. For STM32U3 devices, set mandatory OB:
 - i. TZEN = 1
 - ii. RDP = 0x55
 - iii. SRAM2_RST = 0
 - b. For STM32H5 devices, set mandatory OB:
 - i. TZEN = 1
 - ii. product state = PROVISIONING
 - iii. SRAM2_RST = 0
 - iv. SRAM2_ECC = 0
 - v. SRAM3_ECC = 1
 - vi. SRAM1_3_4_5_RST = 1 for STM32H5Fx, SRAM1_3_RST = 1 for STM32H553
 - c. Get the encrypted RSSe KW binary from the X-Cube RSSe package and download it into SRAM download area.
 - d. Write the address of the encrypted RSSe KW binary to be installed at the address expected by the descriptor.
 - e. Set the RSSCMD value in download area and the address expected by the descriptor "input address".
 - f. Set PC to RSSLIB_Set RSSCMD function call or send BL special command for RSSCMD setting.
 2. Get RSSe KW installation status.
 3. Write descriptor parameters for key wrapping.
 - a. Input address to input structure
 - b. Output address to expected output structure
 4. Write input structure at the expected address.
 5. Call RSSLIB_RSSeCall API with RSSE_KEYWRAP_ECC_CHIP_PRIVATE_KEY_ID.
 6. Read the output structure at the expected address.
- The steps are detailed in the following parts of this document.

3.2.2 STM32 descriptor

The STM32 system flash memory embeds a structure called STM32 descriptor. This structure provides host information regarding the STM32 device and the way the host can communicate with RSS.

The following table gives the C structure that describes the STM32 descriptor fields.

Table 3. stm32_descriptor_t description

Field name	Type	Description
Version	uint32_t	Descriptor version
CertificateAddr	uint32_t	Address of the chip certificate
AvailableRamStartAddr	uint32_t	Start address of the available SRAM for host
AvailableRamEndAddr	uint32_t	End address of the available SRAM for host
RsseLoadAddr	uint32_t	Address where to find address of the RSSe to install
RsseMaxSize	uint32_t	Maximum size of an RSSe
RsseParamAddr	uint32_t	Address where to find the input parameter address for the API call
RsseParamMaxSize	uint32_t	Maximum size of an RSSe parameter
RsseResultAddr	uint32_t	Address where to find RSSe call result value
Reserved	uint32_t[7]	Reserved

Field name	Type	Description
RSSLIBApiTable	RSSLIB_STM32xxApiTable_t	RSSLIB API table within the RSSLIB system flash memory area

The address of STM32 descriptor field is provided within STM32 system memory at a specific address tied to the product.

Table 4. STM32 descriptor address per product

Product lines	STM32 descriptor address
STM32U3	0x0BF99000
STM32H5Fx	0x0BFB6000
STM32H553	0x0BFAE000

3.2.3

RSS library API description

The root secure services library provides services for both secure and nonsecure firmware.

These secure services are split between secure firmware that can only call secure services and the nonsecure firmware that can only call nonsecure service.

The address of secure services is provided within a specific area named RSSLIB_STM32xxApiTable within STM32 system memory at a specific address tied to the product. RSSLIB_STM32xxApiTable address per product.

Table 5. RSSLIB_STM32xxApiTable address per product

Product lines	RSSLIB_STM32xxApiTable address
STM32U3	0x0BF99040
STM32H5Fx	0x0BFB6040
STM32H553	0x0BFAE040

The following table gives the C structure that describes the RSSLIB_STM32xxApiTable dedicated to the supported STM32 microcontrollers.

Table 6. RSSLIB_STM32ApiTable_t structure description description for STM32U3 devices

Field name ⁽¹⁾	Type	Callable Attribute ⁽²⁾	Description
RSSLIB_RSSeCall	RSSLIB_RSSeCall_t	NS	Gateway to call RSSe API
RSSLIB_Call	RSSLIB_Call_t	NS	Gateway to call RSSLIB API
Reserved	uint32_t[8]	NA	NA
RSSLIB_sec_CloseExitHDP	RSSLIB_sec_CloseExitHDP_t	S	Close and exit from flash memory HDP area jumping on passed vector table address (More details in product reference manual.)
RSSLIB_sec_CloseExitHDP_Ext	RSSLIB_sec_CloseExitHDPExt_t	S	Close and exit from flash memory HDPEExt area jumping on passed vector table address (More details in product reference manual.)

1. Each field of RSSLIB_STM32xxApiTable_t is a pointer to a function.

2. NS: nonsecure callable only; S: secure callable only

Table 7. RSSLIB_STM32xxApi_Table_t structure description for STM32H5 devices

Field name ⁽¹⁾	Type	Callable attribute ⁽²⁾	Description
RSSLIB_RSSeCall	RSSLIB_RSSeCall_t	NS	Gateway to call RSSe API
RSSLIB_Call	RSSLIB_Call_t	NS	Gateway to call RSS_lib API
RSSLIB_SetRSSCMD	RSSLIB_SetRSSCMD_t	NS	Set value to RSS command
RSSLIB_SetSecOB	RSSLIB_SetSecOB_t	NS	-
RSSLIB_GetSecOB	RSSLIB_GetSecOB_t	NS	-
RSSLIB_GetRssVersion	RSSLIB_GetRssVersion_t	NS	-
RSSLIB_GetCertificate	RSSLIB_GetCertificate_t	NS	-
RSSLIB_GetCertificateSize	RSSLIB_GetCertificateSize_t	NS	-
reserved0	reserved_func_t	Deprecated NscRsslibDisableBootLock	-
reserved1	reserved_func_t	Deprecated NscRsslibEnableBootLock	-
RSSLIB_DataProvisioning	RSSLIB_DataProvisioning_t	NS	-
NSSLIB_JumpHDPLv2	NSSLIB_JumpHDPLvx_t	NS	-
NSSLIB_JumpHDPLv3	NSSLIB_JumpHDPLvx_t	NS	-
NSSLIB_Puflnit	NSSLIB_Puflnit_t	NS	-
RSSLIB_Sec_JumpHDPLv2	RSSLIB_Sec_JumpHDPLv2_t	S	-
RSSLIB_Sec_JumpHDPLv3	RSSLIB_Sec_JumpHDPLv3_t	S	-
RSSLIB_Sec_JumpHDPLv3NS	RSSLIB_Sec_JumpHDPLv3NS_t	S	-
RSSLIB_Sec_Puflnit	RSSLIB_Sec_Puflnit_t	S	-

1. Each field of `RSSLIB_STM32xxApiTable_t` is a pointer to a function.

2. • *NS: nonsecure callable only*
• *S: secure callable only*

The `RSSLIB_RSSeCall` & `RSSLIB_Call` fields listed in the above table are function types described in the following section.

By default, nonsecure callable services are not defined within a nonsecure callable SAU region. Hence, the user has no SAU parameters nor region to set in order to deny such secure services to nonsecure firmware.

3.2.4 RSSLIB API function types

This section provides the function types in C for RSSLIB API not described within product reference manual and that are relevant for KW.

Note: All addresses given as example in this section are relevant for STM32U385 devices. They have to be adapted for other devices depending on the download area boundaries.

3.2.4.1 RSSLIB_Call

Service description: call a RSSLIB service using the command identifier (CmdID) of the RSSLIB service

Input parameter: `payload_addr`, type `RSSLIB_RSSECmd_t`

Output parameter: parameter stored in the address pointed by `STM32_descriptor->RsseResultAddr`. For example, if `STM32_descriptor->RsseResultAddr = 0x3000800C`, the RSSLIB stores the API result at the address pointed by `0x3000800C`. The output parameter type depends on the RSSLIB service called (described in `RSSLIB_SetRSSCMD`).

Return value: `RSSLIB_RSS_SUCCESS`, `RSSLIB_RSS_ERROR`

function type: `typedef uint32_t (*RSSLIB_Call_t) (RSSLIB_RSSECmd_t *payload_addr);`

Table 8. RSSLIB_RSSECmd_t description

Field name	Type	Comment
CmdID	uint32_t	Command ID
InputParamAddr	uint32_t	API input parameter address

3.2.4.2

RSSLIB_RSSECall

Service description: call a RSSE service using the command identifier (CmdID) of the RSSE service

Input parameter: payload_addr, type [RSSLIB_RSSECmd_t](#)

Output parameter: parameter stored in the address pointed by `STM32_descriptor->RsseResultAddr`. For example, if `STM32_descriptor->RsseResultAddr = 0x3000800C`, the RSSE stores the API result at the address pointed by `0x3000800C`. The output parameter type depends on the RSSE service called (described in [RSSE_KEYWRAP_GetVersion](#)).

Return value: `RSSLIB_RSS_SUCCESS`, `RSSLIB_RSS_ERROR`

function type: `typedef uint32_t (*RSSLIB_RSSECall_t) (RSSLIB_RSSECmd_t *payload_addr);`

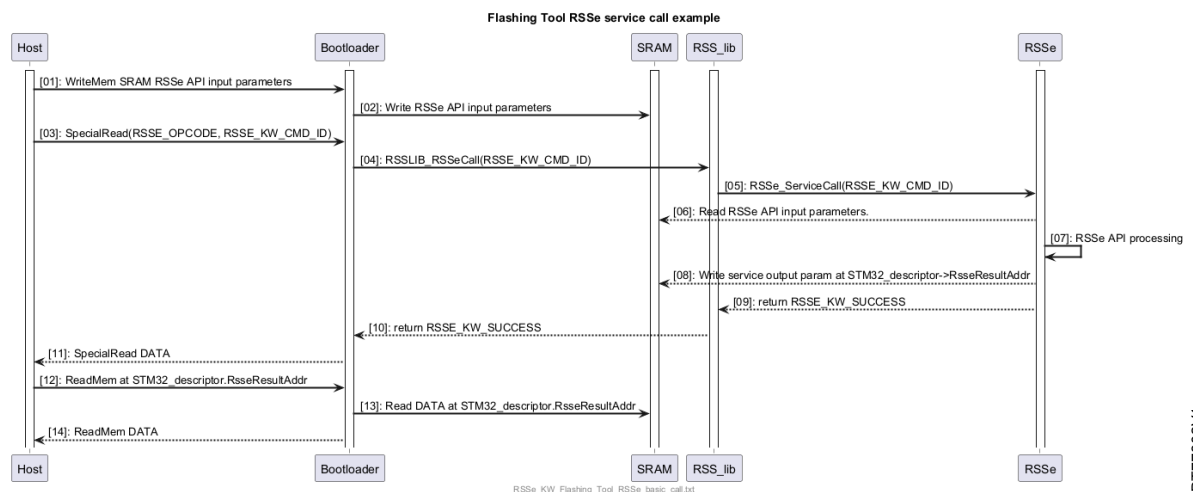
Table 9. RSSLIB_RSSECmd_t description

Field name	Type	Comment
CmdID	uint32_t	Command ID
InputParamAddr	uint32_t	API input parameter address

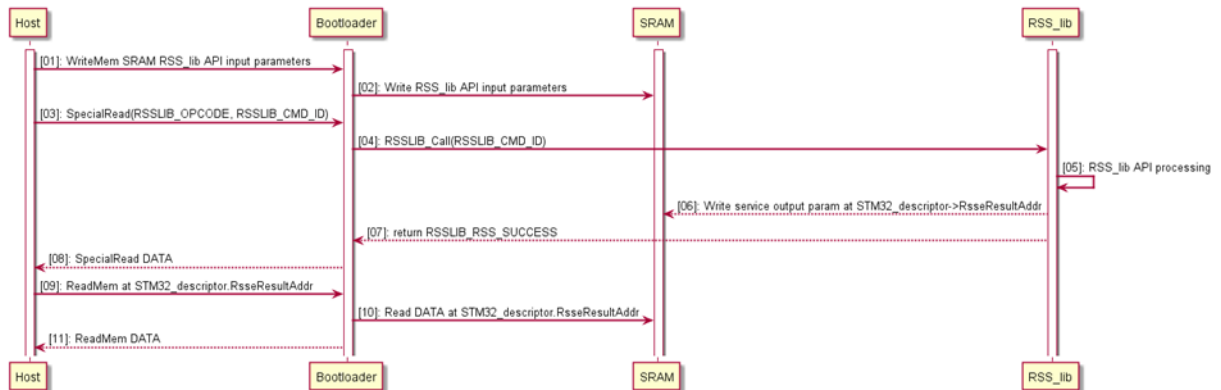
3.3

RSSLIB & RSSE services

For each API, a command ID identifies the called RSSLIB or RSSE service. This command ID is the one mentioned as `RSSLIB_CMD_ID` or `RSSE_KW_CMD_ID` in the following figures.

Figure 3. Host basic RSSE service call


DT77802V1

Figure 4. Host basic RSS_lib service call


3.3.1

RSSLIB_SetRSSCMD

Command ID: RSSLIB_RSSCMD_CMD_ID = 0x00

Service description: User calls RSSLIB_SetRSSCMD in order to request boot from root security service (RSS) on next boot. This request is canceled if bit BOOTLOCK within register FLASH_SECBOOTADD0R is already set.

Input parameter: uint32_t RSSCMD

Output parameter: None

Return value: RSSLIB return status enumeration

Parameter description: see Table 10

Table 10. uint32_t rsscmd values description

Constant name	Value	Comment
RSSLIB_RSSCMD_RSSE_JTAG_BOOTLOADER	0xE00	Request RSS to install RSSE on next STM32 boot

Table 11. RSSLIB return status enumeration

Constant name	Constant value (C hexadecimal notation)
RSSLIB_RSS_SUCCESS	0xEAEAEA
RSSLIB_RSS_ERROR	0xF5F5F5

3.3.2

RSSLIB_GetRssVersion

Command ID: RSSLIB_GET_RSS_VERSION_ID = 0x01

Service description: RSSLIB writes RSS version at address pointed by STM32_descriptor->RsseResultAddr

Input parameter: None

Output parameter: RSSLIB_RSSVersion_t description

Return value: RSSLIB return status enumeration

Table 12. RSSLIB_RSSVersion_t description

Field name	Type	Comment
Patch	uint8_t	Bug fix
Minor	uint8_t	Backward compatible changes: public API deprecated improvements, for new API functions

Field name	Type	Comment
Major	uint8_t	Backward incompatible changes: public API modification
Reserved	uint8_t	-

3.3.3 RSSe_KEYWRAP_GetVersion

Command ID: RSSE_KEYWRAP_GET_VERSION_CMD_ID = 0x02

Service description: RSSe writes RSSe KW version at address pointed by STM32_descriptor->RsseResultAddr.

Input parameter: None

Output parameter: [RSSe_Version_t](#) description

Return value: see [RSSe_Version_t](#) description.

RSSe_Version_t

Table 13. RSSe_Version_t description

Field name	Type	Comment
Patch	uint8_t	Bug fix
Minor	uint8_t	Backward compatible changes: public API deprecated improvements, for new API functions
Major	uint8_t	Backward incompatible changes: public API modification
Reserved	uint8_t	-

Table 14. RSSe_KW return status enumeration

Constant name	Constant value (C hexadecimal notation)
RSSE_KW_SUCCESS	0xEAEAEAEA
RSSE_KW_ERROR	0xFFFF4DA36U
RSSE_KW_OB_INIT_ERROR	0xFFEA3D79U
RSSE_KW_SERVICE_INIT_ERROR	0xFFEAE56U
RSSE_KW_CRC_ERROR	0xF5F5E00EU
RSSE_KW_NO_HW_CRYPT0	0xF5F5A0AAU
RSSE_KW_WRONG_INPUT_PARAM_CONTEXT	0xF5F50E0EU
RSSE_KW_WRONG_INPUT_PARAM_SELECTION	0xF5F5E0E0U
RSSE_KW_EXPORT_PUBLIC_KEY_ECC_ERROR	0xF5F58080U
RSSE_KW_WRONG_INPUT_PARAM_KEY_SEL	0xF5F58008U

3.3.4 RSSe_KEYWRAP_GetStatus

Command ID: RSSE_KEYWRAP_GET_STATUS_CMD_ID = 0x01

Service description: RSSe writes KW status at STM32_descriptor->RsseResultAddr.

Input parameter: None

Output parameter: uint32_t, KW status is described in [Table 15](#).

Return value: [RSSe_KW return status enumeration](#)

Table 15. RSSe KW status enumeration

Constant name	Constant value (C hexadecimal notation)
KW_STATUS_INITIALISED	0xEAEA0808U
KW_STATUS_NOT_INITIALISED	0xF5F50808U

3.3.5

RSSe_KEYWRAP_WrapEccCpvkKey

Command ID: RSSE_KEYWRAP_ECC_CHIP_PRIVATE_KEY_ID = 0x06

Service description: This service returns the wrapped DUA private key container at STM32_descriptor→RsseResultAddr.

Input parameter: InputKeyWrapEccChipPrivateKey_t

Output parameter: BinaryWrappedChipEccPrivateKey_t

Return value: RSSe_KW return status enumeration

The input parameters are detailed below in Table 16 to Table 25.

Table 16. InputKeyWrapEccChipPrivateKey_t for STM32U3 devices

Field name	type	Comment
ECCChipPrivateKeyID	EccChipPrivateKeyID_t	ID of the chip private key
WrappingKeySel	WrappingKeySel_t	Selection of the key to wrap the chip private key
Reserved	uint8_t[1]	4 bytes alignment
Context	WrapUnwrapContext_t	Context to wrap chip private key
Reserved	uint32_t[10]	Reserved for future use
Crc	uint32_t	CRC on above fields

Table 17. InputKeyWrapEccChipPrivateKey_t for STM32H5 devices

Field name	type	Comment
ECCChipPrivateKeyID	EccChipPrivateKeyID_t	ID of the chip private key
Reserved	uint8_t[3]	4 bytes alignment
Context	WrapUnwrapContext_t	Context to wrap chip private key
Reserved	uint32_t[10]	Reserved for future use
Crc	uint32_t	CRC on above fields

Table 18. EccChipPrivateKeyID_t for STM32U3 devices

Constant name	Constant value (C hexadecimal notation)
KW_ECC_CPVK_DUA2_FU	0x00U
KW_ECC_CPVK_DUA2_LU	0x01U

Table 19. EccChipPrivateKeyID_t for STM32H5 devices

Constant name	Constant value (C hexadecimal notation)
KW_ECC_CPVK_DUA_USER	0x03U
KW_ECC_CPVK_DUA_USLU	0x04U

Table 20. WrappingKeySel_t

Constant name	Constant value (C hexadecimal notation)
KW_WRAPPING_KEY_DHUK	0x00U

Table 21. WrapUnwrapContext_t for STM32U3 devices

Field name	type	Comment
KeyUsage	KeyUsage_t	Indicate for what purpose the key is used
Reserved	uint8_t	Reserved for future use
SecAttr	SecAttr_t	Key used in secure or nonsecure domain
PrivAttr	PrivAttr_t	Key used in privileged or nonprivileged mode (only privileged mode available)
Reserved	uint8_t	Reserved for future use

Table 22. WrapUnwrapContext_t for STM32H5 devices

Field name	type	Comment
WrappingKeySel	WrappedKeyId_t	Selection of the key to wrap the chip
KeyUsage	KeyUsage_t	Indicate for what purpose the key is used
Hdpl	uint8_t	HDP level for which the key will be used. It can be either HDPL2 value (0x8A) or HDPL3 value (0x6F)
SecAttr	SecAttr_t	Key used in secure or nonsecure domain
PrivAttr	PrivAttr_t	Key used in privileged or nonprivileged mode (only privileged mode available)
Reserved	uint8_t[3]	Reserved for future use
EpochSel	uint32_t	Define which epoch counter is selected for wrapping. Only epoch counter force to zero value (0xFFFF277D1U) is supported

Table 23. KeyUsage_t

Constant name	Constant value (C hexadecimal notation)
KW_KEY_ECC_USAGE_SIGN	0x00U
KW_KEY_ECC_USAGE_SCALAR_MUL	0x01U

Table 24. SecAttr_t

Constant name	Constant value (C hexadecimal notation)
KW_SEC_ATTR_SECURE	0x00U
KW_SEC_ATTR_NON_SECURE	0xFAU (only available for STM32U3 devices)

Table 25. PrivAttr_t

Constant name	Constant value (C hexadecimal notation)
KW_PRIV_ATTR_PRIVILEGED	0x00U

The output container is composed of a header and a payload described in [Table 26](#) to [Table 31](#).

Table 26. BinaryWrappedChipEccPrivateKey_t

Field name	Type	Comment
Header	BinaryHeader_t for STM32U3 devices or BinaryHeader_t for STM32H5 devices	Header of the output container, containing elements to use the wrapped key
Payload	EccWrappedChipPrivateKeyPayload_t	Payload of the output container, containing the wrapped key

Table 27. BinaryHeader_t for STM32U3 devices

Field name	Type	Comment
WrappedKeyId	WrappedKeyId_t	Identifier of the key
PayloadSize	uint32_t	Payload size in bytes
AdditionalData	AdditionalData_t	Data used to unwrap the wrapped key
RSSeVersion	RSSe_version_t	RSSe KW version
Reserved	uint32_t[7]	Reserved for future usage.
Crc	uint32_t	CRC on above fields and payload

Table 28. BinaryHeader_t for STM32H5 devices

Field name	Type	Comment
WrappedKeyId	WrappedKeyId_t	Identifier of the key
PayloadSize	uint32_t	Payload size in bytes
AdditionalData	AdditionalData_t	Data used to unwrap the wrapped key
RSSeVersion	RSSe_version_t	RSSe KW version
Context	WrapUnwrapContext_t	Context to wrap chip private key
Reserved	uint32_t[4]	Reserved for future usage
Crc	uint32_t	CRC on above fields and payload

Table 29. WrappedKeyId_t

Field name	Type	Comment
ServiceId	uint8_t	Identifier of the RSSe service call (for future use)
KeyUsageld	KeyUsage_t	Key usage identifier
KeySizeId	uint8_t	Key size identifier, 0x00 for ECC curve SECP256R1
Reserved	uint8_t	4 bytes alignment

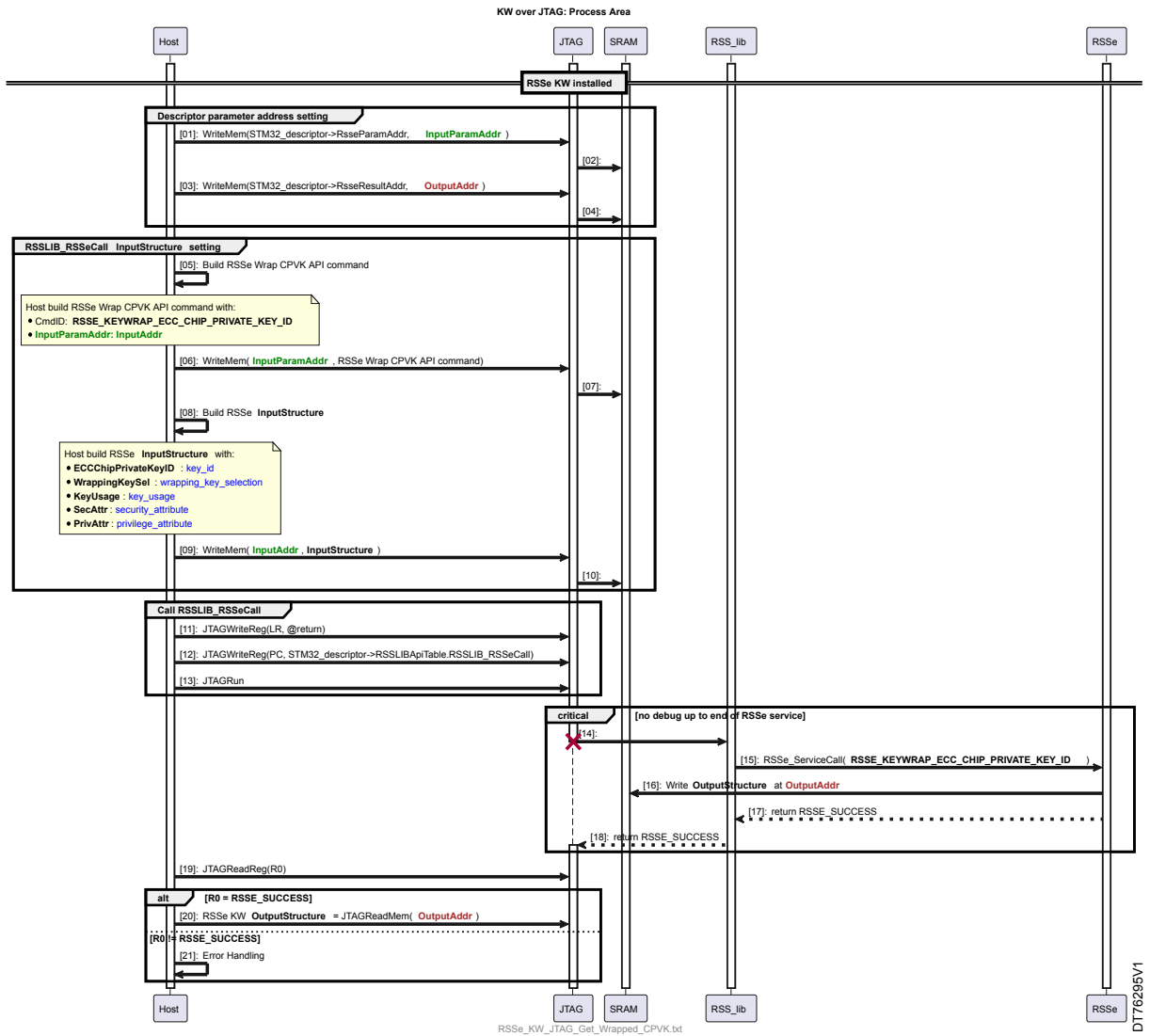
Table 30. AdditionalData_t

Field name	Type	Comment
IV	uint32_t[4]	AES GCM initialization vector
Tag	uint32_t[4]	AES GCM tag
AESMode	uint8_t	AES mode identifier, 0x02 for AES GCM
Reserved	uint8_t[3]	4 bytes alignment

Table 31. EccWrappedChipPrivateKeyPayload_t

Field name	Type	Comment
WrappedPrivateKey	uint8_t[32]	Wrapped Private Key

3.3.6 Dynamic description of key wrap command

Figure 5. KW installation by JTAG: Get Wrapped DUA private key


3.4 Appendix A: kw.h file for STM32U3 devices

```

/**
 * *****
 * @file    kw.h
 * @author  MCD Application Team
 * @brief   key wrapping interface module.
 * *****
 * @attention
 *
 * Copyright (c) 2024 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file

```

```

* in the root directory of this software component.
* If no LICENSE file comes with this software, it is provided AS-IS.
*
*****
*/

#ifndef KW_H
#define KW_H
#include "rsslib.h"

/* Exported constants -----*/
/* KeyWrap error codes */
#define RSSE_KW_SUCCESS (RSSLIB_RSS_SUCCESS)
#define RSSE_KW_ERROR 0xFFFF4DA36U /* Generic value for error during key wrapping */
#define RSSE_KW_OB_INIT_ERROR 0xFFEA3D79U /* Wrong OB programmed for RSSE initialization */
#define RSSE_KW_SERVICE_INIT_ERROR 0xFFEAEE56U /* Error during service call */
#define RSSE_KW_CRC_ERROR 0xF5F5E00EU /* CRC check error on input or output parameters */
#define RSSE_KW_NO_HW_CRYPT0 0xF5F5A0AAU /* No HW crypto on this sample */
#define RSSE_KW_WRONG_INPUT_PARAM_CONTEXT 0xF5F50E0EU /* Context selected not supported */
#define RSSE_KW_WRONG_INPUT_PARAM_SELECTION 0xF5F5E0E0U /* key size ID or Curve ID not supported */
#define RSSE_KW_EXPORT_PUBLIC_KEY_ECC_ERROR 0xF5F58080U /* ECC export public key error */
#define RSSE_KW_WRONG_INPUT_PARAM_KEY_SEL 0xF5F58008U /* wrap AES key, wrapping key selection not supported */

/* Key sizes definition for output container */
/* ECC SECP256R1 */
#define RSSE_ECC_256R1_CPVK_SIZE 32U

/* Additionnal data */
#define RSSE_AES_GCM_IV_SIZE 4U
#define RSSE_AES_GCM_TAG_SIZE 4U
#define RSSE_AES_CBC_IV_SIZE 4U

/* Exported types -----*/
enum rsse_kw_service_id
{
    RSSE_KEYWRAP_SERVICE_ID_BASIC = (0U),
    RSSE_KEYWRAP_GET_STATUS_ID = (1U),
    RSSE_KEYWRAP_GET_VERSION_ID = (2U),
    RSSE_KEYWRAP_ECC_CHIP_PRIVATE_KEY_ID = (6U),
    RSSE_KW_MAX_ID
};

typedef enum
{
    KW_WRAP_KEY_DHUK = (0U), /* Use DHUK to wrap */
} WrappingKeySel_t;

typedef enum
{
    AES_MODE_ECB = (0U), /* AES ECB Mode to wrap AES key, (no IV, no Tag) */
    AES_MODE_CBC, /* AES CBC Mode to wrap AES key, (IV, no Tag) */
    AES_MODE_GCM /* AES GCM mode to wrap asymmetric key, (IV, Tag) */
} AESMode_t;

typedef struct
{
    uint32_t IV[RSSE_AES_GCM_IV_SIZE]; /* IV to unwrap the wrapped key */
    uint32_t Tag[RSSE_AES_GCM_TAG_SIZE]; /* Tag used to verify the unwrapped key */
    AESMode_t AESMode; /* AES mode to wrap the key */
    uint8_t Reserved[3U]; /* 4 bytes alignment */
} AdditionalData_t;

typedef enum
{

```



```

KW_KEY_ECC_USAGE_SIGN = (0U), /* ECC Key used to compute signature */
W_KEY_ECC_USAGE_SCALAR_MUL, /* ECC Key used to compute scalar multiplication */
} KeyUsage_t;

typedef enum
{
    KW_SEC_ATTR_SECURE = (0U), /* Wrap key for usage in secure domain */
} SecAttr_t;

typedef enum
{
    KW_PRIV_ATTR_PRIVILEGED = (0U), /* Wrap key for usage in privileged domain */
} PrivAttr_t;

typedef struct
{
    KeyUsage_t KeyUsage; /* Indicate for what purpose the key will be used */
    uint8_t Hdpl; /* HDPL level using the key Parameter. Not used: 0xFE */
    SecAttr_t SecAttr; /* Key used in Secure or Non-Secure domain */
    PrivAttr_t PrivAttr; /* Key used in Privileged or Non-Privileged mode */
    uint8_t CpuId; /* On multi cores platform, indicates which CPU uses the key. Not used
: 0xFE */
} WrapUnwrapContext_t;

typedef enum
{
    KW_ECC_CPVK_DUA2_FU = (0U), /* Chip private key DUA2 FU */
    KW_ECC_CPVK_DUA2_LU /* Chip private key DUA2 LU */
} EccChipPrivateKeyID_t;

typedef struct
{
    EccChipPrivateKeyID_t EccChipPrivateKeyID; /* ID of the Chip Private key */
    WrappingKeySel_t WrappingKeySel; /* Selection of the key to wrap the Chip Private
key */
    uint8_t Reserved[1U]; /* 4 bytes alignment */
    WrapUnwrapContext_t Context; /* Context to wrap Chip Private key */
    uint32_t Reserved[10U]; /* Reserved for futur usage */
    uint32_t Crc; /* CRC on above fields and key */
} InputKeyWrapEccChipPrivateKey_t;

/* Output container structures */
typedef union
{
    EccChipPrivateKeyID_t EccCpvkSizeId;
} KeySizeId_t;

typedef struct
{
    uint8_t ServiceId; /* Taken from rsse_kw_service_id values, only 0x06 available now */
    KeyUsage_t KeyUsageId; /* Depending on the key type, indicates the usage of the key */
    KeySizeId_t KeySizeId; /* From the union of the different key size identifiers */
    uint8_t Reserved; /* 4 bytes alignment */
} WrappedKeyId_t;

typedef struct
{
    WrappedKeyId_t WrappedKeyID; /* Identifier of the key */
    uint32_t PayloadSize; /* Payload size in bytes */
    AdditionalData_t AdditionalData; /* Data used to unwrap the wrapped key */
    RSSE_version_t RSSEVersion; /* RSSE KW version */
    uint32_t Reserved[7U]; /* Reserved for future usage */
    uint32_t Crc; /* CRC on above fields and payload */
} BinaryHeader_t;

typedef struct
{
    uint8_t WrappedPrivateKey [RSSE_ECC_256R1_CPVK_SIZEB];
}EccWrappedChipPrivateKeyPayload_t;

```

```
typedef struct
{
    BinaryHeader_t Header;
    EccWrappedChipPrivateKeyPayload_t Payload;
} BinaryWrappedChipEccPrivateKey_t;
```

3.5 Appendix B: kw.h for STM32H5Fx/553

```
/**
 * *****
 * @file    kw.h
 * @author  MCD Application Team
 * @brief   key wrapping interface module.
 * *****
 * @attention
 *
 * Copyright (c) 2024 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 * *****
 */

#ifndef KW_H
#define KW_H
#include "rssl.h"

/* Exported constants -----*/
/* KeyWrap error codes */
#define RSSE_KW_SUCCESS (RSSLIB_RSS_SUCCESS)
#define RSSE_KW_ERROR 0xFFFF4DA36U /* Generic value for error during key wrapping */
#define RSSE_KW_OB_INIT_ERROR 0xFFEA3D79U /* Wrong OB programmed for RSSE initialization */
#define RSSE_KW_SERVICE_INIT_ERROR 0xFFEAEE56U /* Error during service call */
#define RSSE_KW_CRC_ERROR 0xF5F5E00EU /* CRC check error on input or output parameters */
#define RSSE_KW_NO_HW_CRYPTO 0xF5F5A0AAU /* No HW crypto on this sample */
#define RSSE_KW_WRONG_INPUT_PARAM_CONTEXT 0xF5F50E0EU /* Context selected not supported */
#define RSSE_KW_WRONG_INPUT_PARAM_SELECTION 0xF5F5E0E0U /* key size ID or Curve ID not supported */
#define RSSE_KW_EXPORT_PUBLIC_KEY_ECC_ERROR 0xF5F58080U /* ECC export public key error */
#define RSSE_KW_WRONG_INPUT_PARAM_KEY_SEL 0xF5F58008U /* wrap AES key, wrapping key selection not supported */

/* Key sizes definition for output container */
/* ECC SECP256R1 */
#define RSSE_ECC_256R1_CPVK_SIZE_B 32U

/* Additionnal data */
#define RSSE_AES_GCM_IV_SIZE_W 4U
#define RSSE_AES_GCM_TAG_SIZE_W 4U
#define RSSE_AES_CBC_IV_SIZE_W 4U

/* Force to zero ID for EpochSel configuration */
#define EPOCH_FORCE_TO_ZERO_ID 0xFFFF277D1U

/* Exported types -----*/

enum rsse_kw_service_id
{
    RSSE_KEYWRAP_SERVICE_ID_BASIC = (0U),
    RSSE_KEYWRAP_GET_STATUS_ID = (1U),
    RSSE_KEYWRAP_GET_VERSION_ID = (2U),
    RSSE_KEYWRAP_ECC_CHIP_PRIVATE_KEY_ID = (6U),
}
```

```

    RSSE_KW_MAX_ID
};

typedef enum
{
    KW_WRAPPING_KEY_DHUK = (0U), /* Use DHUK to wrap */
} WrappingKeySel_t;

typedef enum
{
    AES_MODE_ECB = (0U), /* AES ECB Mode to wrap AES key, (no IV, no Tag) */
    AES_MODE_CBC,        /* AES CBC Mode to wrap AES key, (IV, no Tag) */
    AES_MODE_GCM          /* AES GCM mode to wrap asymmetric key, (IV, Tag) */
} AESMode_t;

typedef struct
{
    uint32_t IV[RSSE_AES_GCM_IV_SIZE_W]; /* IV to unwrap the wrapped key */
    uint32_t Tag[RSSE_AES_GCM_TAG_SIZE_W]; /* Tag used to verify the unwrapped key */
    AESMode_t AESMode; /* AES mode to wrap the key */
    uint8_t Reserved[3U]; /* 4 bytes alignment */
} AdditionalData_t;

typedef struct
{
    uint8_t *pAESWrappedKey; /* Pointer to the wrapped AES key */
    AdditionalData_t AdditionalData; /* Data used to unwrap the wrapped key */
} WrappingKey_t;

typedef enum
{
    KW_KEY_ECC_USAGE_SIGN = (0U), /* ECC Key used to compute signature */
    KW_KEY_ECC_USAGE_SCALAR_MUL, /* ECC Key used to compute scalar multiplication */
} KeyUsage_t;

typedef enum
{
    KW_SEC_ATTR_SECURE = (0U), /* Wrap key for usage in secure domain */
} SecAttr_t;

typedef enum
{
    KW_PRIV_ATTR_PRIVILEGED = (0U), /* Wrap key for usage in privileged domain */
} PrivAttr_t;

typedef struct
{
    WrappingKeySel_t WrappingKeySel; /* Selection of the key used to wrap the key */
    KeyUsage_t KeyUsage; /* Indicate for what purpose the key will be used */
    uint8_t HdpL; /* HDPLevel using the key Parameter */
    SecAttr_t SecAttr; /* Key used in Secure or Non-Secure domain */
    PrivAttr_t PrivAttr; /* Key used in Privileged or Non-Privileged mode */
    uint8_t CpuId; /* On multi cores platform, indicates which CPU uses the
key. Not used: 0xFE */
    uint8_t Reserved[2U]; /* 4 bytes alignment */
    uint32_t EpochSel;
} WrapUnwrapContext_t;

typedef enum
{
    KW_ECC_CPVK_DUA_USER = (3U), /* Chip private key DUA USER */
    KW_ECC_CPVK_DUA_USLU = (4U), /* Chip private key DUA USLU */
} EccChipPrivateKeyID_t;

typedef struct
{
    EccChipPrivateKeyID_t EccChipPrivateKeyID; /* ID of the Chip Private key */
    uint8_t Reserved[3U]; /* 4 bytes alignment */
    WrapUnwrapContext_t Context; /* Context to wrap Chip Private key */
    WrappingKey_t WrappingKey; /* Wrapped key used to wrap the Chip Private key

```

```

    /*
        uint32_t          Crc;                      /* CRC on above fields and key */
    } InputKeyWrapEccChipPrivateKey_t;

    /* Output container structures */
    typedef union
    {
        EccChipPrivateKeyID_t ChipPrivateKeyId;
    } KeySizeId_t;

    typedef struct
    {
        uint8_t ServiceId;          /* Taken from rsse_kw_service_id values from 0x03 to 0x06 */
        KeyUsage_t KeyUsageId;      /* Depending on the key type, indicates the usage of the key */
        KeySizeId_t KeySizeId;      /* From the union of the different key size identifiers */
        uint8_t Reserved;           /* 4 bytes alignment */
    } WrappedKeyId_t;

    typedef struct
    {
        WrappedKeyId_t WrappedKeyId;      /* Identifier of the key */
        uint32_t PayloadSize;              /* Payload size in bytes */
        AdditionalData_t AdditionalData;   /* Data used to unwrap the wrapped key */
        RSSE_version_t RSSEVersion;        /* RSSE KW version */
        WrapUnwrapContext_t Context;        /* Context to wrap Chip Private key */
        uint32_t Reserved[4U];              /* Reserved for future usage */
        uint32_t Crc;                      /* CRC on above fields and payload */
    } BinaryHeader_t;

    typedef struct
    {
        uint8_t WrappedPrivateKey [RSSE_ECC_256R1_CPVK_SIZE_B];
    } EccWrappedChipPrivateKeyPayload_t;

    typedef struct
    {
        BinaryHeader_t Header;
        EccWrappedChipPrivateKeyPayload_t Payload;
    } BinaryWrappedChipEccPrivateKey_t;

```

Revision history

Table 32. Document revision history

Date	Version	Changes
21-Mar-2025	1	Initial release.
03-Mar-2026	2	Added Section 2.2: Wrapped key usage constraints .
08-Jul-2026	3	Updated: • Table 1. Applicable products • References • Section 1: General information • Section 1.2: Glossary • Section 2.2: Wrapped key usage constraints • Section 2.3.1: Internal key wrapped • Section 2.4: RSSE KW constraints • Section 3.2.1: Key wrap flow overview • Section 3.2.2: STM32 descriptor • Section 3.2.4: RSSLIB API function types • Section 3.2.4.1: RSSLIB_Call • Section 3.2.4.2: RSSLIB_RSSECall • Section 3.2.3: RSS library API description • Section 3.3.5: RSSE_KEYWRAP_WrapEccCpvkKey • Section 3.3.6: Dynamic description of key wrap command • Section 3.4: Appendix A: kw.h file for STM32U3 devices Added: • Section 3.5: Appendix B: kw.h for STM32H5Fx/553

Contents

1	General information	2
1.1	References.....	2
1.2	Glossary.....	2
2	STM32 RSSe Key Wrapping.....	4
2.1	RSSe KW principles overview.....	4
2.2	Wrapped key usage constraints	4
2.3	RSSe KW features	4
2.3.1	Internal key wrapped.....	4
2.4	RSSe KW constraints.....	4
3	Interface.....	5
3.1	Cube Programmer interface.....	5
3.2	Tool/ RSSLIB interface	5
3.2.1	Key wrap flow overview.....	5
3.2.2	STM32 descriptor	6
3.2.3	RSS library API description.....	7
3.2.4	RSSLIB API function types	8
3.3	RSSLIB & RSSe services.....	9
3.3.1	RSSLIB_SetRSSCMD.....	10
3.3.2	RSSLIB_GetRssVersion	10
3.3.3	RSSe_KEYWRAP_GetVersion.....	11
3.3.4	RSSe_KEYWRAP_GetStatus.....	11
3.3.5	RSSe_KEYWRAP_WrapEccCpvkKey	12
3.3.6	Dynamic description of key wrap command	15
3.4	Appendix A: kw.h file for STM32U3 devices	15
3.5	Appendix B: kw.h for STM32H5Fx/553.....	18
	Revision history	21
	List of tables	23
	List of figures.....	24

List of tables

Table 1.	Applicable products	1
Table 2.	List of documents.	2
Table 3.	stm32_descriptor_t description	6
Table 4.	STM32 descriptor address per product	7
Table 5.	RSSLIB_STM32xxApiTable address per product	7
Table 6.	RSSLIB_STM32ApiTable_t structure description description for STM32U3 devices	7
Table 7.	RSSLIB_STM32xxApi_Table_t structure description for STM32H5 devices	8
Table 8.	RSSLIB_RSSCmd_t description	9
Table 9.	RSSLIB_RSSeCmd_t description.	9
Table 10.	uint32_t rsscmd values description.	10
Table 11.	RSSLIB return status enumeration	10
Table 12.	RSSLIB_RSSVersion_t description	10
Table 13.	RSSe_Version_t description	11
Table 14.	RSSe_KW return status enumeration	11
Table 15.	RSSe KW status enumeration	12
Table 16.	InputKeyWrapEccChipPrivateKey_t for STM32U3 devices	12
Table 17.	InputKeyWrapEccChipPrivateKey_t for STM32H5 devices	12
Table 18.	EccChipPrivateKeyID_t for STM32U3 devices	12
Table 19.	EccChipPrivateKeyID_t for STM32H5 devices	12
Table 20.	WrappingKeySel_t	13
Table 21.	WrapUnwrapContext_t for STM32U3 devices	13
Table 22.	WrapUnwrapContext_t for STM32H5 devices	13
Table 23.	KeyUsage_t	13
Table 24.	SecAttr_t	13
Table 25.	PrivAttr_t	13
Table 26.	BinaryWrappedChipEccPrivateKey_t	14
Table 27.	BinaryHeader_t for STM32U3 devices	14
Table 28.	BinaryHeader_t for STM32H5 devices	14
Table 29.	WrappedKeyId_t	14
Table 30.	AdditionalData_t	14
Table 31.	EccWrappedChipPrivateKeyPayload_t	15
Table 32.	Document revision history	21

List of figures

Figure 1.	Cube Programmer interface with user	5
Figure 2.	Detailed programming tool interface with user.	5
Figure 3.	Host basic RSSe service call	9
Figure 4.	Host basic RSS_lib service call.	10
Figure 5.	KW installation by JTAG: Get Wrapped DUA private key	15

IMPORTANT NOTICE – READ CAREFULLY

STMicroelectronics NV and its subsidiaries ("ST") reserve the right to make changes, corrections, enhancements, modifications, and improvements to ST products and/or to this document at any time without notice.

In the event of any conflict between the provisions of this document and the provisions of any contractual arrangement in force between the purchasers and ST, the provisions of such contractual arrangement shall prevail.

The purchasers should obtain the latest relevant information on ST products before placing orders. ST products are sold pursuant to ST's terms and conditions of sale in place at the time of order acknowledgment.

The purchasers are solely responsible for the choice, selection, and use of ST products and ST assumes no liability for application assistance or the design of the purchasers' products.

No license, express or implied, to any intellectual property right is granted by ST herein.

Resale of ST products with provisions different from the information set forth herein shall void any warranty granted by ST for such product.

If the purchasers identify an ST product that meets their functional and performance requirements but that is not designated for the purchasers' market segment, the purchasers shall contact ST for more information.

ST and the ST logo are trademarks of ST. For additional information about ST trademarks, refer to www.st.com/trademarks. All other product or service names are the property of their respective owners.

Information in this document supersedes and replaces information previously supplied in any prior versions of this document.

© 2026 STMicroelectronics – All rights reserved